exclurad_py: user manual

Exclusive meson electroproduction with exact radiative corrections

V. Kubarovsky
Jefferson Lab

August 23, 2026

Abstract

`exclurad_py` generates exclusive meson electroproduction events $ep \to e' h X$ for π^0, η, ρ, ϕ and J/ψ with exact first-order QED radiative corrections on the EXCLURAD framework, and with pre-vertex external radiation in the target. This manual is what you need to run it: how to get the code, what the directory tree means, every field of the run card, how the radiative correction is extracted from a pair of runs, and what to change to add a new structure-function model. It is deliberately short; the physics, the validation and the evidence are in the companion note.

Contents

1 Getting the code **1**

2 Directory tree **2**

3 The run card **2**

4 Running **3**

5 Extracting the radiative correction **4**

6 Adding a structure-function model **4**

7 Checking a sample **6**

1 Getting the code

The repository is `https://github.com/vkubarovsky/exclurad_py`. Production runs must use a tagged commit, never a branch tip: the tag is written into every output file and is how a sample is traced back to the code that made it.

```
git clone -b feature/exclurad_py git@github.com:vkubarovsky/exclurad_py.git
cd exclurad_py
git checkout prod-pi0-v1 # a tag, not a branch
git describe --tags # must echo the tag back
```

Requirements are only `python3` and `numpy`; `matplotlib` is needed for the plotting tools, not for generation. Versions in use range from python 3.9 with numpy 1.23 on the JLab farm to python 3.14 with numpy 2.4 on macOS, and both give statistically identical samples; the cross-machine comparison is in the companion note.

Deploying to a machine without GitHub access. `production/deploy.sh` copies the tree and writes `exclurad_py/DEPLOYED_VERSION` beside the package, which the provenance code reads when `.git` is absent. Use it only when a clone is impossible: a real clone lets `git describe` verify the version *on the machine*, while the file merely asserts it.

2 Directory tree

```
exclurad_py/ the repository
  exclurad_py/ the python package (import root)
    core/ kinematics, Born, rc_factor -- the physics
    models/ structure-function models, one class each
    generator/ sampling, tables, decay, LUND writer
    topology.py which hadron is detected; sign conventions
    runcard.py the run-card parser
  cards/ run cards, one per channel
  production/ run scripts, validation and plotting tools
  validation/ reports and reference data
  doc/ this manual and the note
```

Output is written *outside* the repository, into a directory you name in the run card. The recommended layout, produced by `production/run_prod_v1.sh`:

```
<data>/pi0_v1/
  README.md box, code version, checks the version rests on
  card_rad.txt the cards actually used
  card_born.txt
  part_runner.py written by the run script; nothing lives outside
  cyc01/ cyc02/ ... one directory per cycle
    pi0/<date>_<E>_<mode>_v1/lund/*.lund
    logs/
  STOP create this file to stop the loop cleanly
```

Cycles are independent: each has its own seed, and samples from different cycles and different machines add up. To stop a running production, `touch STOP` and the loop finishes its cycle and exits. Do not kill the processes.

3 The run card

Plain text, one value per line, `!` starts a comment. The *order of the lines is the format* — there are no keywords. Blank and comment-only lines are skipped. Fortran exponents (`1.0d-4`) are accepted.

#	field	meaning
1	model	`<channel>.<vintage>`, e.g. `pi0.vpk2021`. Not an integer channel code: several vintages of the same channel give visibly different RC, and the name goes into the output header. `--models` lists them.
2	ebeam	beam energy, GeV
3	nev	events to write
4	iborn	1 = Born only, 0 = full RC
5	W2 range	GeV^2
6	Q2 range	GeV^2
7	t range	GeV^2, both negative
8	vcut	$v_{\mathrm{cut}} = M_X^2 - m_U^2$, GeV^2; 0 = generate to the kinematic limit
9	t_nucl	GeV^2, π^0/η models only; ignored for vector mesons
10	ihel	$+1$, -1, or 0 = draw the helicity per event
11	seed	
12	idecay	1 = decay the meson, 0 = leave it
13	output root	`<channel>/<date>_<E>_<mode>_v1/` is created under it
optional, may be omitted from the end		
14	y range	ν/E
15	xB range	
16	iacc	acceptance preset: `none` or `jpsi`
17	vmin	soft/hard separator, GeV^2; 0 = choose automatically
18	itarg	1 = pre-vertex external radiator in the target
19	vz range	cm, the target cell, e.g. `-7.5 -2.5`

Two fields that are easy to confuse. `vcut` is *physics*: the exclusivity cut of the analysis ($0.20\ \mathrm{GeV}^2$ for π^0 and η, 1.20 for J/ψ). `vmin` is *arithmetic*: the internal soft/hard separator of the Bardin–Shumeiko decomposition, conventionally 10^{-4}. Nothing physical may depend on `vmin`, and the note shows that nothing does.

`itarg` and the meaning of `vcut`. With the radiator on, `vcut` is applied in *true* variables, at the event's own reduced beam energy. An analysis cuts on the *reconstructed* missing mass, using the nominal beam energy, and about 0.8 % of the events will fail that cut. This is correct and the sample is complete: external energy loss can only raise the reconstructed inelasticity, $v_{\mathrm{rec}} - v_{\mathrm{true}} = 2\,\Delta E\,(E_X - p_{X,z}) \geq 0$, so events leave the window and never enter it.

4 Running

One part of a run:

```
export PYTHONPATH=$PWD
export TABLE_SERIAL=1 ETA_PROPOSAL_CAP=5 EXACT_ACCEPT=1 EXPO_TAIL=1
python3 -c "from exclurad_py.run import run; run('cards/rad_pi0.txt')"
```

For anything larger use `production/run_prod_v1.sh`, which splits the work over cores, writes the README and the cards into the output directory, and loops until a deadline:

```
production/run_prod_v1.sh <code_dir> <out_dir> <seed_base> <ncores> <deadline_epoch> <
    label>
```

On a batch farm use `production/slurm_submit.sh`, which submits the table-building part first and the rest as an array depending on it. Building the η table once and letting the array read it from cache is not an optimisation: launched together, the tasks race for the same file.

5 Extracting the radiative correction

Run the same card twice, changing only `iborn`, and the correction is the ratio of the two samples bin by bin:

$$\delta(\text{bin}) \;=\; \frac{N_{\text{rad}}(\text{bin})/N_{\text{rad}}}{N_{\text{born}}(\text{bin})/N_{\text{born}}} \;\times\; \frac{\sigma_{\text{obs}}}{\sigma_{\text{Born}}}, \qquad \frac{\Delta\delta}{\delta} = \sqrt{\frac{1}{n_{\text{rad}}} + \frac{1}{n_{\text{born}}}}. \tag{1}$$

Both samples come from the same generator, so the ratio tests the radiative machinery and nothing else. Born events are about ten times cheaper, so make the Born sample five times larger and its contribution to the error becomes negligible.

`production/report_run.py` does this for twelve variables at once:

```
python3 production/report_run.py <run_dir> -c pi0
```

6 Adding a structure-function model

A model is one class in `exclurad_py/models/`. It states its provenance, its validity range and its topology, and implements one method returning the five structure functions in nb/GeV^2:

```
class MyModel(StructureFunctions):
    name = "pi0.mine2027"
    channel = "pi0"
    version = "2027"
    provenance = "fit to <data>, table N of <note>"
    reference = "arXiv:....."
    validity = dict(W2=(3.2, 16.0), Q2=(0.5, 5.0), t=(-3.0, -0.001))
    topology = baryon_detected(0.1349768, meson="pi0")

    def _sfs(self, W2, Q2, t):
        return dict(sT=..., sL=..., sTT=..., sLT=..., sLTp=...)
```

Register it in `models/__init__.py` and the name works in any card.

Four things to get right. *Topology* decides which hadron is detected, and with it the sign of σ_{LT} and the azimuth offset; do not set those by hand. *Units* are nb/GeV^2 in the Hand convention. *validity* is not bureaucracy: the radiative tail evaluates the model at shifted kinematics, so a fit made on a limited grid *will* be called outside it, and the excursions are counted and reported. And a moment known only as a *fraction* of $\sigma_U = \sigma_T + \varepsilon\sigma_L$ — say $\sigma_{LT'}/\sigma_U = 0.10$ — is declared as `SLTP_OVER_SU = 0.10` rather than written into `_sfs`: the base class then builds it from σ_U at the event's own ε, which needs the beam energy and stays correct when σ_L stops being zero.

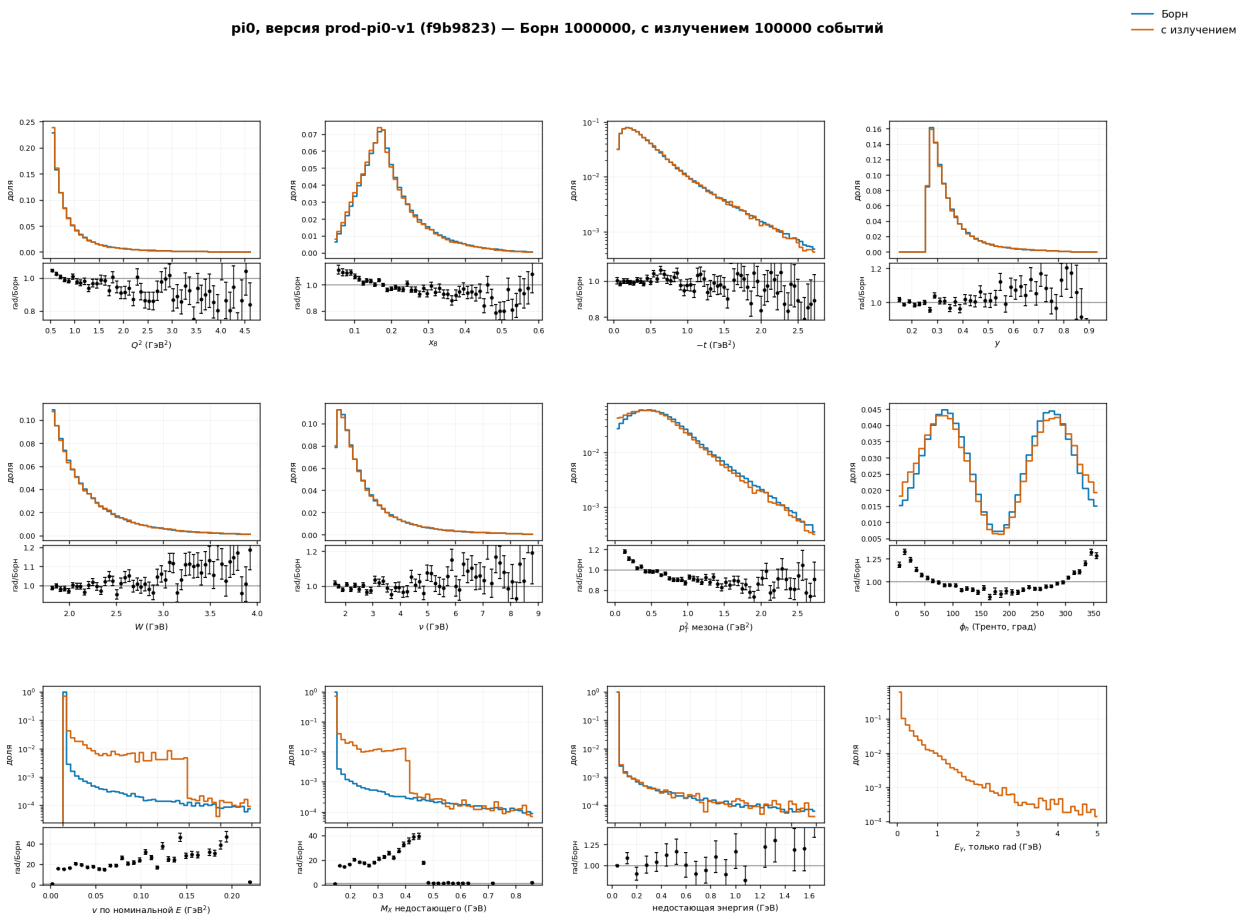


Figure 1: Output of `report_run.py`. Upper half of each cell: the Born and radiative distributions, each normalised to unit area. Lower half: their ratio, which *is* the radiative correction as a function of that variable, measured on the events with no quadrature involved. The strongest dependence is on the azimuth, from 1.25 to 0.85.

7 Checking a sample

tool	what it answers
`check_lund.py`	Is the output internally consistent? Four-momentum, mass shells, decay closure, charge, header.
`report_run.py`	What does the correction look like in twelve variables?
`rc_proof_phi.py`	Does the event density reproduce the exact calculation? Events against quadrature in ϕ.
`delta_profiles.py`	The same in Q^2, W, $\lvert t\rvert$, ϕ.
`soft_limit.py`	Do radiative events with $v \to 0$ become Born?
`compare_machines.py`	Do two machines give the same sample?
`estimate_rate.py`	How long will this card take, before starting?

Two habits worth keeping. Point the tools at the right card: they read `cards/rad_<ch>.txt` unless `CARD` or `CARD_PREFIX` says otherwise, and a sample measured against the wrong card fails silently rather than loudly. And read the `health:` line of every run — it reports majorant violations, which bias an unweighted sample and cannot be repaired afterwards.